



哈尔滨工业大学 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

实验报告

开课学期: 2025 秋季

课程名称: 计算思维与实践

实验名称: 音乐播放器

实验性质: 设计型

实验学时: 2 地点: T2608

学生学号:

学生姓名:

评阅教师:

报告成绩:

实验与创新实践教育中心制

2025 年 12 月

一、实验目的

1. 掌握单向链表的基本概念及常见操作；
2. 理解链表在动态数据管理中的应用场景；
3. 掌握文件读写操作，实现数据的持久化存储；
4. 提升程序设计与综合调试能力。

二、实验内容

设计并实现一个简单的音乐播放器歌单管理系统，使用单向链表存储歌曲信息，实现以下功能：

1. 从文件中读取歌曲信息，并建立歌单链表；
2. 支持用户添加新的歌曲信息；
3. 支持用户删除指定歌曲；
4. 支持按歌曲名查找并“播放”歌曲；
5. 显示当前歌单列表；
6. 将更新后的歌单信息写回文件。

三、系统设计（40 分）

3.1 数据结构设计

本系统主要使用结构体和指针来实现单向链表。

1. 歌曲节点结构体 (Song):

用于存储单首歌曲的具体信息以及指向下一首歌曲的指针。

```
typedef struct Song {  
    int id;                // 歌曲序号  
    char title[100];       // 歌曲标题  
    char artist[50];       // 歌手名  
    char filepath[300];    // 文件路径  
    struct Song* next;     // 指向下一节点的指针（单链表核心）  
} Song;
```

2. 播放列表管理器结构体 (PlaylistManager):

用于维护整个链表的状态，方便函数传参。

```
typedef struct PlaylistManager {  
    Song* head;           // 链表头指针，指向第一首歌  
    Song* tail;           // 链表尾指针，指向最后一首歌（用于尾部插入）  
    Song* current;        // 当前选中/播放的歌曲指针  
    int song_count;       // 链表中歌曲总数  
} PlaylistManager;
```

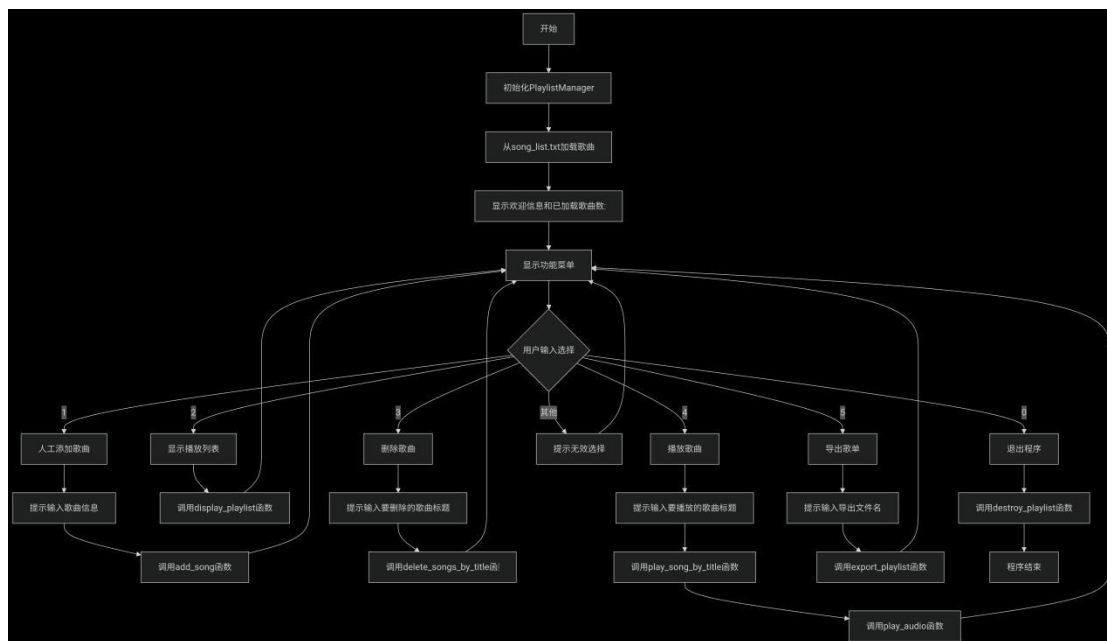
链表组织形式：带头尾指针的单向链表。

逻辑关系：head 指向首节点，节点间通过 next 依次相连，tail 指向最后一个节点，末节点的 next 为 NULL。

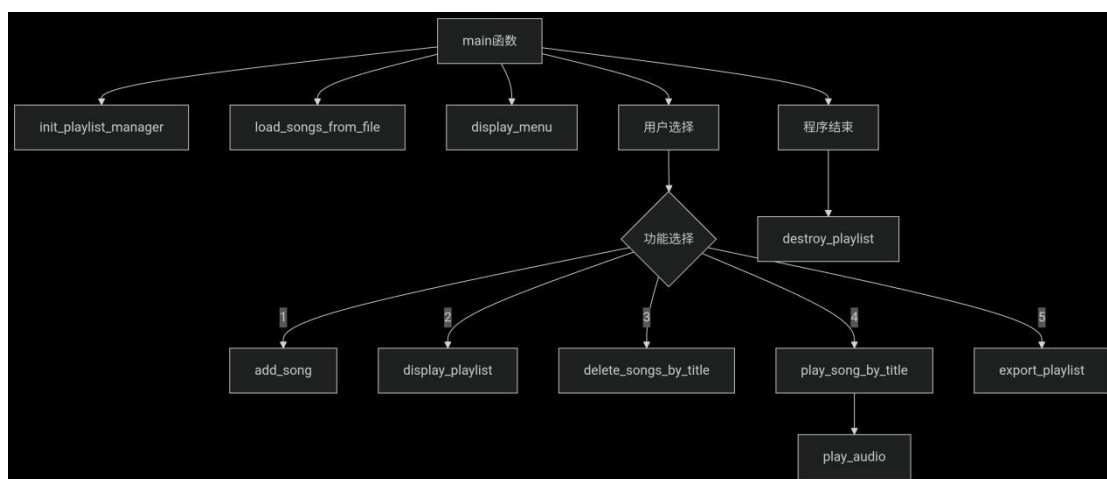
3.2 程序总体结构设计

程序采用模块化设计，主函数 main 负责初始化和交互循环，具体功能封装在各子函数中。

1. 初始化阶段：调用 init_playlist_manager 初始化指针，调用 load_songs_from_file 加载文件数据。
2. 交互循环：使用 do-while 循环显示菜单，根据用户输入的数字（switch-case）调用对应的功能函数（添加、删除、显示等）。
3. 结束阶段：调用 destroy_playlist 释放链表所有节点的内存，避免内存泄漏。
4. 程序运行流程



5. 函数调用关系



四、函数设计（40 分）

4.1 函数接口定义

列出程序中主要函数的函数名、参数说明、返回值及其功能描述。

函数名	参数说明	返回值	功能说明
init_playlist_manager	PlaylistManager* manager	void	初始化管理器，将头尾指针置空，计数归零。
load_songs_from_file	PlaylistManager* manager, const char* filename	int (1 成功, 0 失败)	从指定文本文件按行读取数据并构建链表。

函数名	参数说明	返回值	功能说明
add_song	manager, title, artist, filepath	void	创建新节点并插入到链表尾部。
display_playlist	PlaylistManager* manager	void	遍历链表并格式化输出所有歌曲信息。
delete_songs_by_title	PlaylistManager* manager, title	int	根据标题查找并删除节点，释放内存。
play_song_by_title	PlaylistManager* manager, title	int	查找歌曲并调用系统播放命令。
export_playlist	PlaylistManager* manager, filename	int	将当前链表数据写入文件。
destroy_playlist	PlaylistManager* manager	void	遍历并释放链表所有节点的内存。

4.2 主要函数功能说明

分别说明以下函数的设计思路和实现逻辑：

1. 从文件读取歌曲信息并建立链表的函数：load_songs_from_file

（1）实现思路：逐行读取文件，按逗号分割字段，调用 add_song 插入链表。

（2）伪代码：

打开文件

while 未到文件末尾：

 读取一行

 分割为 title, artist, filepath

 if 三个字段均有效：

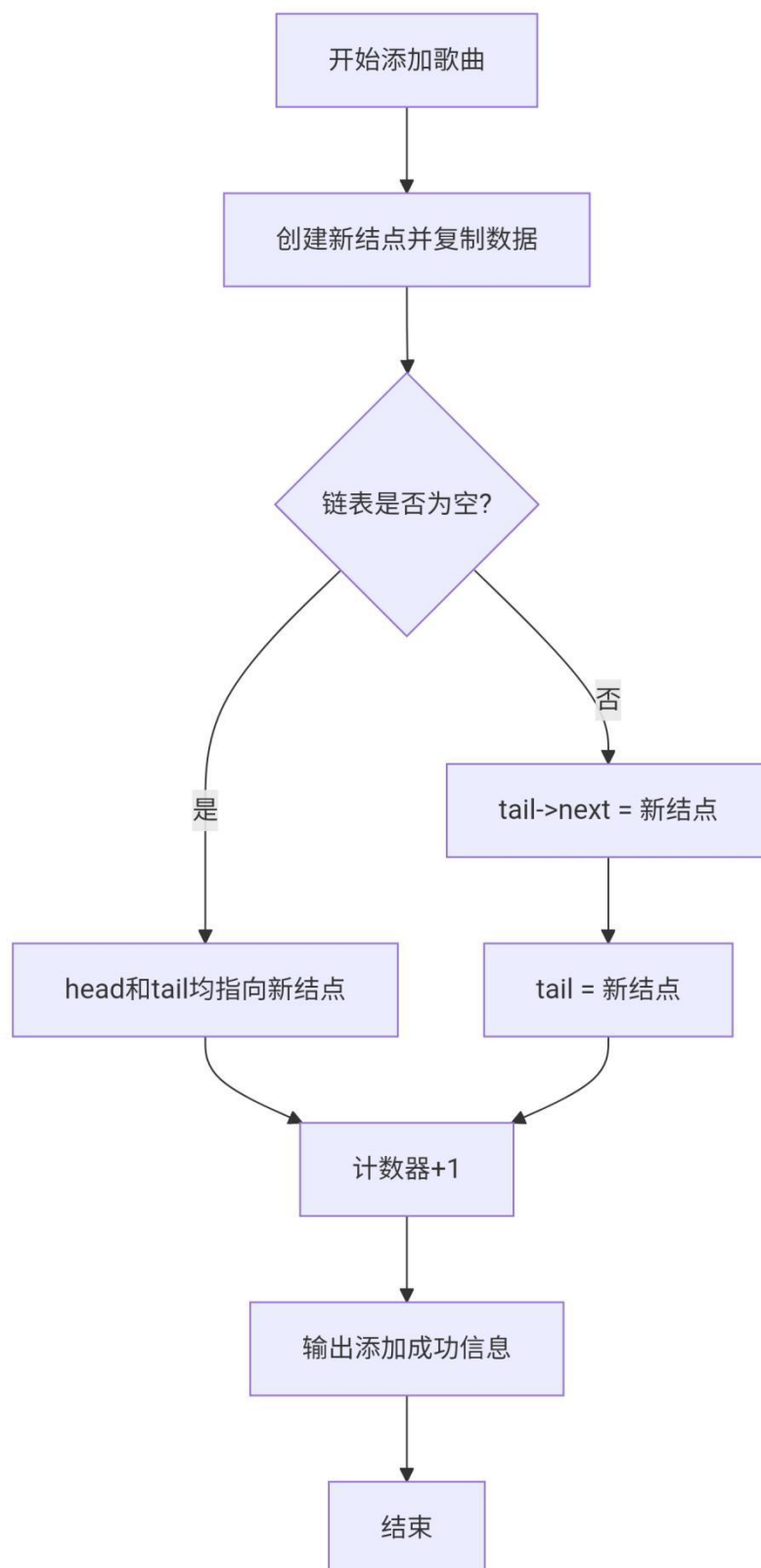
 add_song(manager, title, artist, filepath)

关闭文件

2. 添加歌曲信息的函数：add_song

（1）实现思路：创建新节点并填充歌曲信息，根据链表是否为空采取不同策略将节点插入链表尾部。

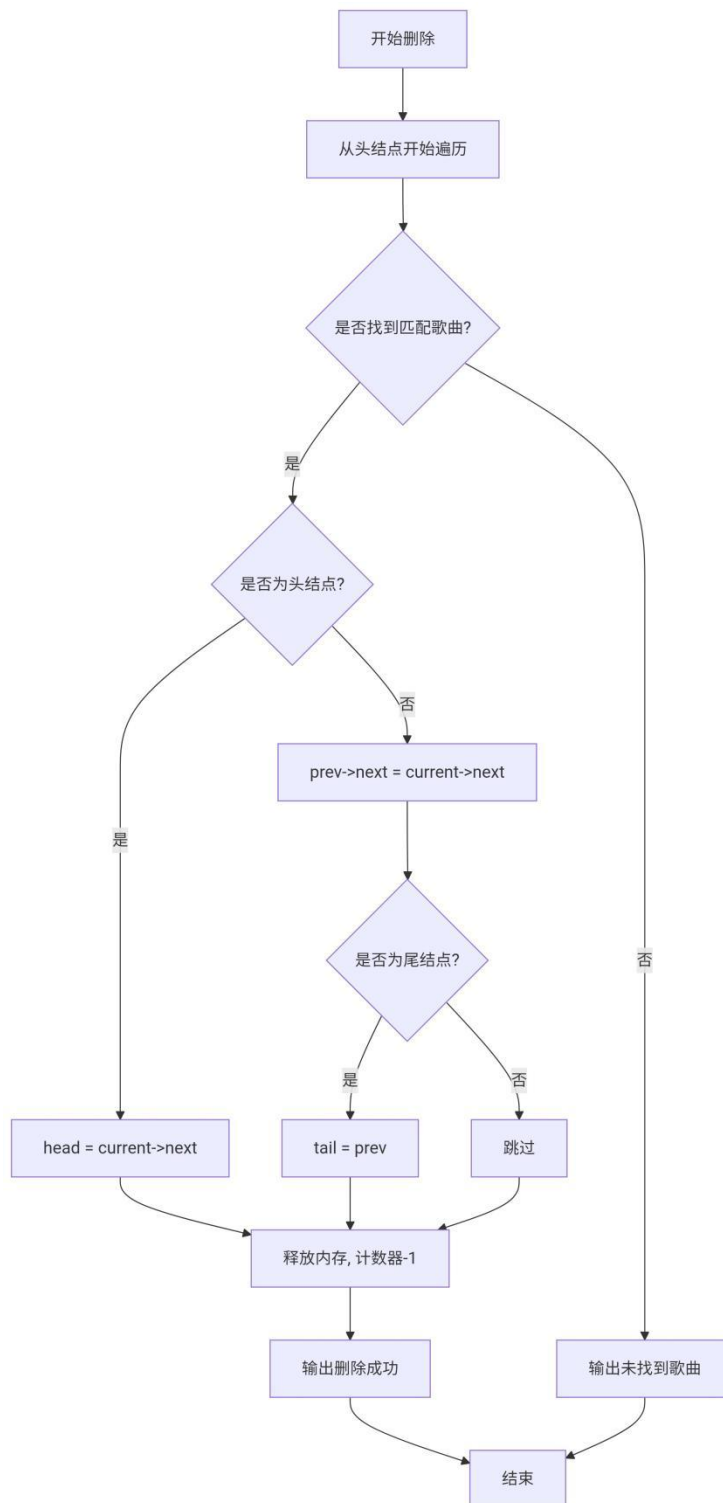
（2）流程图：



3. 删除指定歌曲的函数：delete_songs_by_title

(1) 实现思路：遍历链表查找匹配歌曲标题的节点，找到后根据节点在链表中的位置（头节点、中间节点、尾节点）采用不同的删除策略。

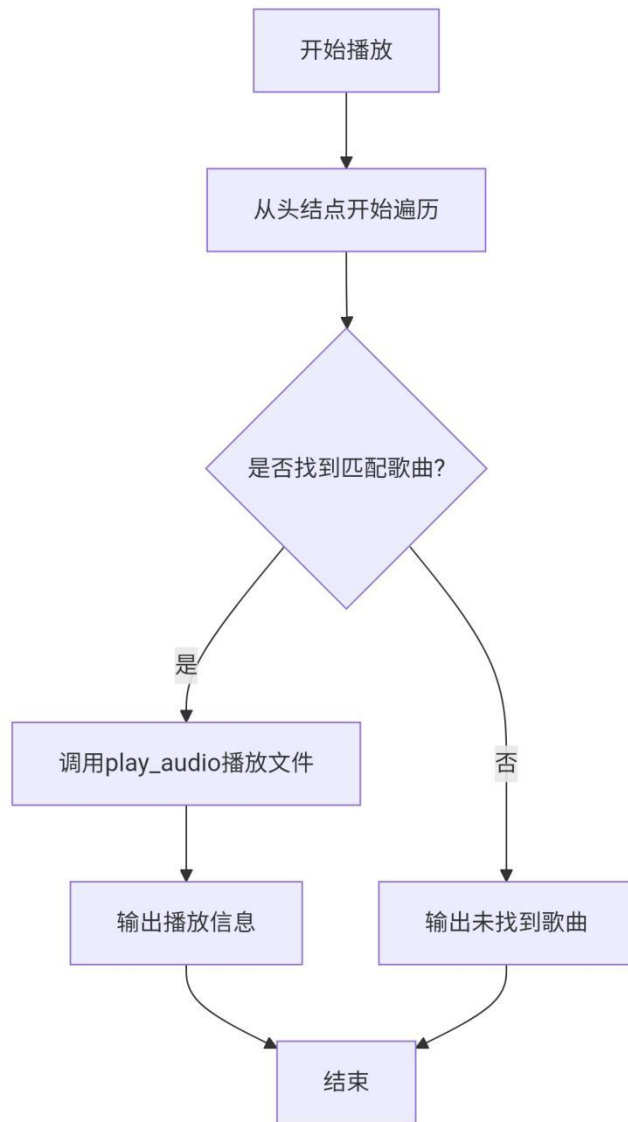
(2) 流程图:



4. 按歌曲名查找并播放歌曲的函数: `play_song_by_title`

(1) 实现思路: 遍历链表查找指定标题的歌曲, 找到后调用系统命令播放对应的音频文件。

(2) 流程图:



5. 显示歌单列表的函数：display_playlist

(1) 实现思路：遍历链表并格式化输出每首歌曲的详细信息，包括序号、标题、歌手和文件路径。

(2) 伪代码：

```
if head == NULL:
```

```
    输出“播放列表为空”
```

```
else:
```

```
    current = head
```

```
    while current != NULL:
```

```
        输出 current 的 title, artist, filepath
```

```
        current = current->next
```

6. 将链表数据写回文件的函数：export_playlist

(1) 实现思路：以追加模式打开目标文件，遍历链表将每首歌曲的信息按格式写入文件，实现数据持久化存储。

(2) 伪代码：

打开文件（追加模式）

current = head

while current != NULL:

 写入 current 的 title, artist, filepath

 current = current->next

关闭文件

输出保存成功

五、实验结果与测试（20 分）

5.1 测试用例设计

用例标号	用例名称	前提条件	用例描述	测试步骤	期望输出	类型
TC1	加载默认歌单	目录下存在 song_list.txt (2 首歌)	测试程序启动时能否正确加载文件	1. 运行程序	提示“已加载 2 首歌曲”；显示菜单	正常值
TC2	显示播放列表	已加载歌单	测试列表显示格式是否对齐	1. 输入菜单选项 '2'	按表格形式打印出 ID, Title, Artist, Filepath	正常值
TC3	添加新歌曲	无	测试尾部插入功能	1. 输入 '1' 2. 输入歌名 "Lemon" 3. 输入歌手 "Kenshi" 4. 输入路径 "musics/Lemon.mp3"	提示“已添加歌曲”；再次显示列表可见第 3 首歌	正常值

用例标号	用例名称	前提条件	用例描述	测试步骤	期望输出	类型
TC4	播放存在的歌曲	列表中有Canon ForElise	测试查找及播放功能	1. 输入 '4' 2. 输入歌名 "Canon"	提示“正在播放Canon”； 弹出播放器窗口	正常值
TC5	删除头节点	列表中有 多首歌	测试逻辑删除 第一首歌 (head 指针更新)	1. 输入 '3' 2. 输入列表第一首歌名	提示“已删除”； 列表第一首变为原来的第二首	边界值
TC6	删除尾节点	列表中有 多首歌	测试逻辑删除 最后一首歌 (tail 指针更新)	1. 输入 '3' 2. 输入列表最后一首歌名	提示“已删除”； 再次添加歌曲应接在倒数第二首后	边界值
TC7	空链表删除	列表为空	测试空链表下 程序是否正常运行	1. 清空列表后输入 '3' 2. 输入任意歌名	提示“列表为空” 或“未找到”；程序 不崩溃	边界值
TC8	删除不存在的歌曲	列表中有 歌曲	测试查找失败 的情况	1. 输入 '3' 2. 输入 "NotExist" 提示“未找到该歌曲”	提示“未找到该歌曲”	异常值
TC9	菜单输入越界	在主菜单 界面	测试菜单选项 输入的容错性	1. 输入菜单选项 '9'	提示“无效选择， 请重新输入”	异常值

5.2 测试结果展示

展示程序运行结果，可通过截图或文字形式说明各功能是否正确实现。

1. 正常值

(1) TC1&TC2

```

F:\实验11&12\实验11&12\Link1_ x + v
已添加歌曲：ForElise-Beethoven
已添加歌曲：Canon-Pachelbel
=== 链表音乐播放器管理器 ===
已加载 2 首示例歌曲

链表音乐播放器管理器
=====
1. 人工添加歌曲
2. 显示播放列表
3. 删除歌曲（按标题）
4. 播放歌曲（按标题）
5. 导出歌单
6. 随机播放歌曲（非必做）
7. 在指定位置添加歌曲（非必做）
8. 清空播放列表（非必做）
0. 退出程序
=====
请选择操作（0-8）：2
1 ForElise          Beethoven          musics\ForElise.mp3
2 Canon              Pachelbel          musics\Canon.mp3

按回车键继续...|

```

(2)TC3

```

F:\实验11&12\实验11&12\Link1_ x + v
8. 清空播放列表（非必做）
0. 退出程序
=====
请选择操作（0-8）：1
请输入歌曲标题：Lemon
请输入作者：Kenshi
请输入歌曲路径：musics/Lemon.mp3

已添加歌曲：Lemon-Kenshi

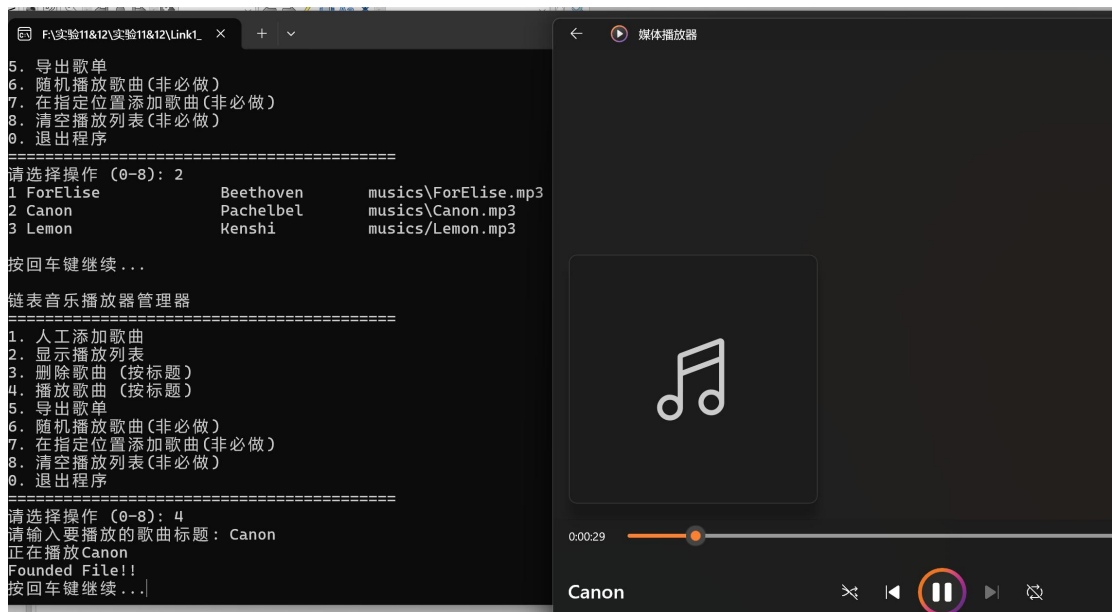
按回车键继续...

链表音乐播放器管理器
=====
1. 人工添加歌曲
2. 显示播放列表
3. 删除歌曲（按标题）
4. 播放歌曲（按标题）
5. 导出歌单
6. 随机播放歌曲（非必做）
7. 在指定位置添加歌曲（非必做）
8. 清空播放列表（非必做）
0. 退出程序
=====
请选择操作（0-8）：2
1 ForElise          Beethoven          musics\ForElise.mp3
2 Canon              Pachelbel          musics\Canon.mp3
3 Lemon              Kenshi              musics/Lemon.mp3

按回车键继续...|

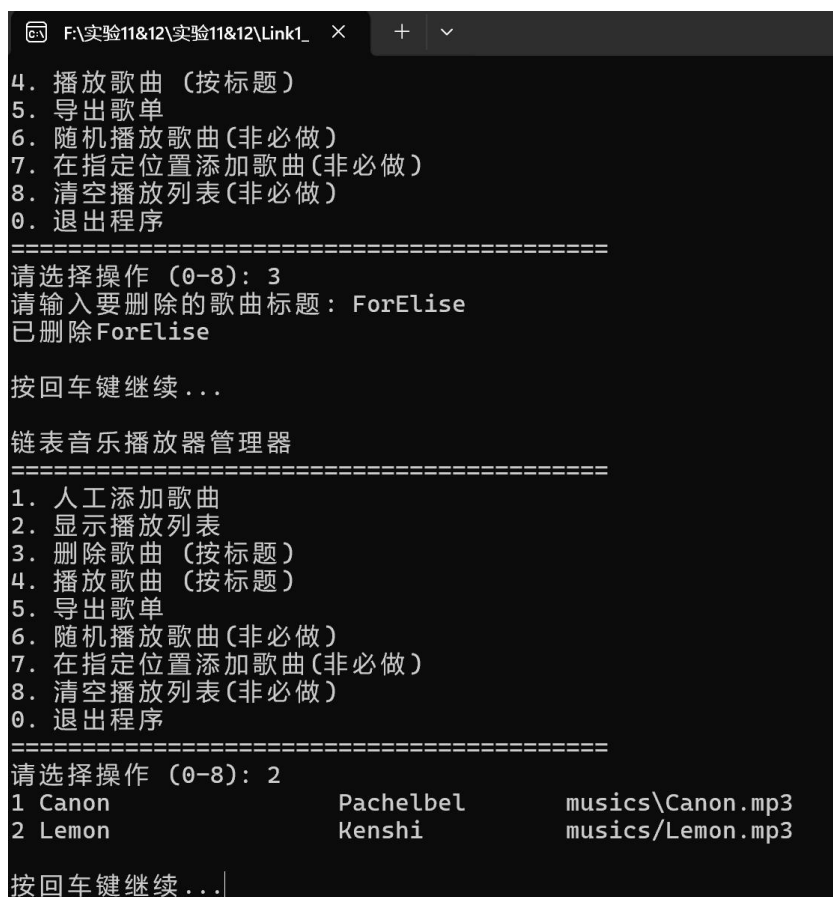
```

(3)TC4



2. 边界值

(4)TC5



(5)TC6

```

F:\实验11&12\实验11&12\Link1_ x + v
3. 删除歌曲 (按标题)
4. 播放歌曲 (按标题)
5. 导出歌单
6. 随机播放歌曲(非必做)
7. 在指定位置添加歌曲(非必做)
8. 清空播放列表(非必做)
0. 退出程序
=====
请选择操作 (0-8): 3
请输入要删除的歌曲标题: Lemon
已删除Lemon

按回车键继续...

链表音乐播放器管理器
=====
1. 人工添加歌曲
2. 显示播放列表
3. 删除歌曲 (按标题)
4. 播放歌曲 (按标题)
5. 导出歌单
6. 随机播放歌曲(非必做)
7. 在指定位置添加歌曲(非必做)
8. 清空播放列表(非必做)
0. 退出程序
=====
请选择操作 (0-8): 2
1 Canon Pachelbel musics\Canon.mp3

按回车键继续...

```

(6)TC7

```

F:\实验11&12\实验11&12\Link1_ x + v
3. 删除歌曲 (按标题)
4. 播放歌曲 (按标题)
5. 导出歌单
6. 随机播放歌曲(非必做)
7. 在指定位置添加歌曲(非必做)
8. 清空播放列表(非必做)
0. 退出程序
=====
请选择操作 (0-8): 2
播放列表为空

按回车键继续...

链表音乐播放器管理器
=====
1. 人工添加歌曲
2. 显示播放列表
3. 删除歌曲 (按标题)
4. 播放歌曲 (按标题)
5. 导出歌单
6. 随机播放歌曲(非必做)
7. 在指定位置添加歌曲(非必做)
8. 清空播放列表(非必做)
0. 退出程序
=====
请选择操作 (0-8): 3
请输入要删除的歌曲标题: Canon
未找到该歌曲

按回车键继续...|

```

3. 异常值

(7)TC8

链表音乐播放器管理器

- ```
=====
1. 人工添加歌曲
2. 显示播放列表
3. 删除歌曲 (按标题)
4. 播放歌曲 (按标题)
5. 导出歌单
6. 随机播放歌曲(非必做)
7. 在指定位置添加歌曲(非必做)
8. 清空播放列表(非必做)
0. 退出程序
=====
```

请选择操作 (0-8): 3

请输入要删除的歌曲标题: NotExist

未找到该歌曲

## (8) TC9

```
F:\实验11&12\实验11&12\Link1_ x + v
2. 显示播放列表
3. 删除歌曲 (按标题)
4. 播放歌曲 (按标题)
5. 导出歌单
6. 随机播放歌曲(非必做)
7. 在指定位置添加歌曲(非必做)
8. 清空播放列表(非必做)
0. 退出程序
=====
请选择操作 (0-8): 2
1 Canon Pachelbel musics\Canon.mp3

按回车键继续...
```

## 链表音乐播放器管理器

- ```
=====
1. 人工添加歌曲
2. 显示播放列表
3. 删除歌曲 (按标题)
4. 播放歌曲 (按标题)
5. 导出歌单
6. 随机播放歌曲(非必做)
7. 在指定位置添加歌曲(非必做)
8. 清空播放列表(非必做)
0. 退出程序
=====
```

请选择操作 (0-8): 9

无效选择, 请重新输入

按回车键继续...

六、问题与解决方法

在实验过程中遇到了以下问题并已解决：

1. 文件读取数量异常（读取 2 行变 3 行）

（1）现象：`song_list.txt` 只有两行数据，但程序显示加载了 3 首歌，且第 3 首是乱码。

（2）原因：原代码使用 `fscanf` 读取时，文件末尾的换行符导致了多一次循环匹配，且未对空行进行过滤。

（3）解决：改用 `fgets` 按行读取，配合 `strcspn` 去除换行符，并增加逻辑判断：只有当一行中包含完整的三个字段（标题、歌手、路径）时才执行添加操作。

2. 程序首次添加歌曲时崩溃

（1）现象：在空链表状态下调用 `add_song` 程序直接闪退。

（2）原因：初始化时 `tail` 为 `NULL`，直接执行 `manager->tail->next` 导致空指针解引用。

（3）解决：在 `add_song` 中增加判断：若 `head` 为空，直接让 `head` 和 `tail` 都指向新节点；否则再通过 `tail->next` 进行链接。

3. 删除歌曲时报错或链表断裂

（1）现象：删除节点后，后续节点丢失或程序报错。

（2）原因：代码中错误地释放了管理器结构体 `free(manager)` 而不是节点 `free(current)`，且删除尾节点时未更新 `tail` 指针。

（3）解决：修正 `free` 的对象为当前节点指针，并完善了删除尾节点时 `tail` 指针前移的逻辑。

4. 列表输出不对齐

（1）现象：长短不一的歌名导致显示列表杂乱。

（2）解决：在 `printf` 中使用 `%-20s` 等格式控制符，指定字段宽度并左对齐，使输出呈现整齐的表格状。

七、课程总结与建议

理论课课时太少而且讲得不够系统，感觉还有很多细节掌握得不好，有很多思路都不会，希望以后不要压缩程设课时。

